

# BUILD AN NGINX TRAFFIC GATEWAY

By Ahmed Tetteh | September 2026

```
127 127.0.0.1 "GET /app/ HTTP/1.1" 200 upstream=127.0.0.1:8082 cache=-
128 127.0.0.1 "GET / HTTP/1.1" 200 upstream=- cache=-
129 127.0.0.1 "GET /app/ HTTP/1.1" 200 upstream=127.0.0.1:8081 cache=-
130 127.0.0.1 "GET / HTTP/1.1" 200 upstream=- cache=-
131 127.0.0.1 "GET /app/ HTTP/1.1" 200 upstream=127.0.0.1:8081 cache=MISS
132 127.0.0.1 "GET / HTTP/1.1" 200 upstream=- cache=-
133 127.0.0.1 "GET /app/ HTTP/1.1" 200 upstream=127.0.0.1:8082 cache=MISS
134 127.0.0.1 "GET / HTTP/1.1" 200 upstream=- cache=-
135 127.0.0.1 "GET /app/ HTTP/1.1" 200 upstream=127.0.0.1:8081 cache=MISS
136 127.0.0.1 "GET / HTTP/1.1" 200 upstream=- cache=-
137 127.0.0.1 "GET /app/ HTTP/1.1" 200 upstream=127.0.0.1:8082 cache=MISS
138 127.0.0.1 "GET / HTTP/1.1" 200 upstream=- cache=-
139 127.0.0.1 "GET /app/ HTTP/1.1" 200 upstream=127.0.0.1:8081 cache=MISS
140 127.0.0.1 "GET / HTTP/1.1" 200 upstream=- cache=-
141 127.0.0.1 "GET /app/ HTTP/1.1" 200 upstream=127.0.0.1:8082 cache=MISS
142 127.0.0.1 "GET / HTTP/1.1" 200 upstream=- cache=-
143 127.0.0.1 "GET /app/ HTTP/1.1" 200 upstream=127.0.0.1:8081 cache=MISS
144 127.0.0.1 "GET /app/ HTTP/1.1" 200 upstream=- cache=HIT
145 127.0.0.1 "GET /app/ HTTP/1.1" 200 upstream=- cache=HIT
146 127.0.0.1 "GET /app/ HTTP/1.1" 200 upstream=- cache=HIT
147 127.0.0.1 "GET /app/ HTTP/1.1" 200 upstream=- cache=HIT
```

# Restoring the Gateway and Balancing Traffic

## Configuring the proxy and upstream servers

So in this step, I'm setting up two backend servers to distribute requests to them using round robin load balancing.

## Verifying requests were distributed across servers

I observed repeated requests being sent to the backend servers on rotation by the Round-robin technique.

8081 and 8082 both appear, so upstream backend\_pool is live rather than a single hardcoded target. Also, both backends send Cache-Control no-store, which nginx honors and refuses to store the response — so you'd still see cache=MISS on every request, never HIT, until I overrode with "proxy\_ignore\_headers Cache-Control" in the /app/ location.

Now setting up caching also masks the load balancing - so once a response is cached, upstream= goes empty on hits, and you'll stop seeing 8081/8082 alternate, making it an expected response to caching, not a regression.

```
127 127.0.0.1 "GET /app/ HTTP/1.1" 200 upstream=127.0.0.1:8082 cache=-
128 127.0.0.1 "GET / HTTP/1.1" 200 upstream=- cache=-
129 127.0.0.1 "GET /app/ HTTP/1.1" 200 upstream=127.0.0.1:8081 cache=-
130 127.0.0.1 "GET / HTTP/1.1" 200 upstream=- cache=-
131 127.0.0.1 "GET /app/ HTTP/1.1" 200 upstream=127.0.0.1:8081 cache=MISS
132 127.0.0.1 "GET / HTTP/1.1" 200 upstream=- cache=-
133 127.0.0.1 "GET /app/ HTTP/1.1" 200 upstream=127.0.0.1:8082 cache=MISS
134 127.0.0.1 "GET / HTTP/1.1" 200 upstream=- cache=-
135 127.0.0.1 "GET /app/ HTTP/1.1" 200 upstream=127.0.0.1:8081 cache=MISS
136 127.0.0.1 "GET / HTTP/1.1" 200 upstream=- cache=-
137 127.0.0.1 "GET /app/ HTTP/1.1" 200 upstream=127.0.0.1:8082 cache=MISS
138 127.0.0.1 "GET / HTTP/1.1" 200 upstream=- cache=-
139 127.0.0.1 "GET /app/ HTTP/1.1" 200 upstream=127.0.0.1:8081 cache=MISS
140 127.0.0.1 "GET / HTTP/1.1" 200 upstream=- cache=-
141 127.0.0.1 "GET /app/ HTTP/1.1" 200 upstream=127.0.0.1:8082 cache=MISS
142 127.0.0.1 "GET / HTTP/1.1" 200 upstream=- cache=-
143 127.0.0.1 "GET /app/ HTTP/1.1" 200 upstream=127.0.0.1:8081 cache=MISS
144 127.0.0.1 "GET /app/ HTTP/1.1" 200 upstream=- cache=HIT
145 127.0.0.1 "GET /app/ HTTP/1.1" 200 upstream=- cache=HIT
146 127.0.0.1 "GET /app/ HTTP/1.1" 200 upstream=- cache=HIT
147 127.0.0.1 "GET /app/ HTTP/1.1" 200 upstream=- cache=HIT
```

# Diagnosing a Planned Gateway Failure

## Creating a controlled 502 Bad Gateway scenario

In this step, I'm configuring Nginx as a reverse proxy by adding a second path at another server on a different port.

## Tracing the request path through the error log

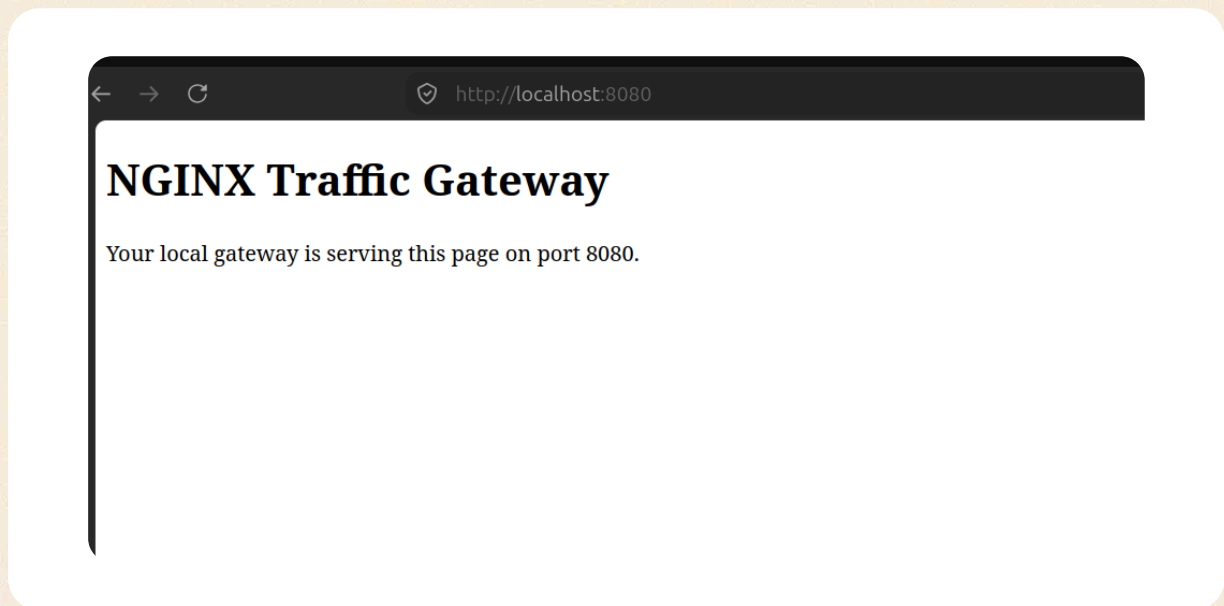
I observed that the browser could reach the gateway on port 8080, while Nginx couldn't reach the new path on port 8999. This is because Nginx is not listening on that port.



# Serving My First NGINX Gateway Page

## Publishing a website through the gateway

In this step , I'm saving Nginx a landing page that returns to the browser on port 8080.



# Building the NGINX Gateway Lab

## Project goals and gateway architecture

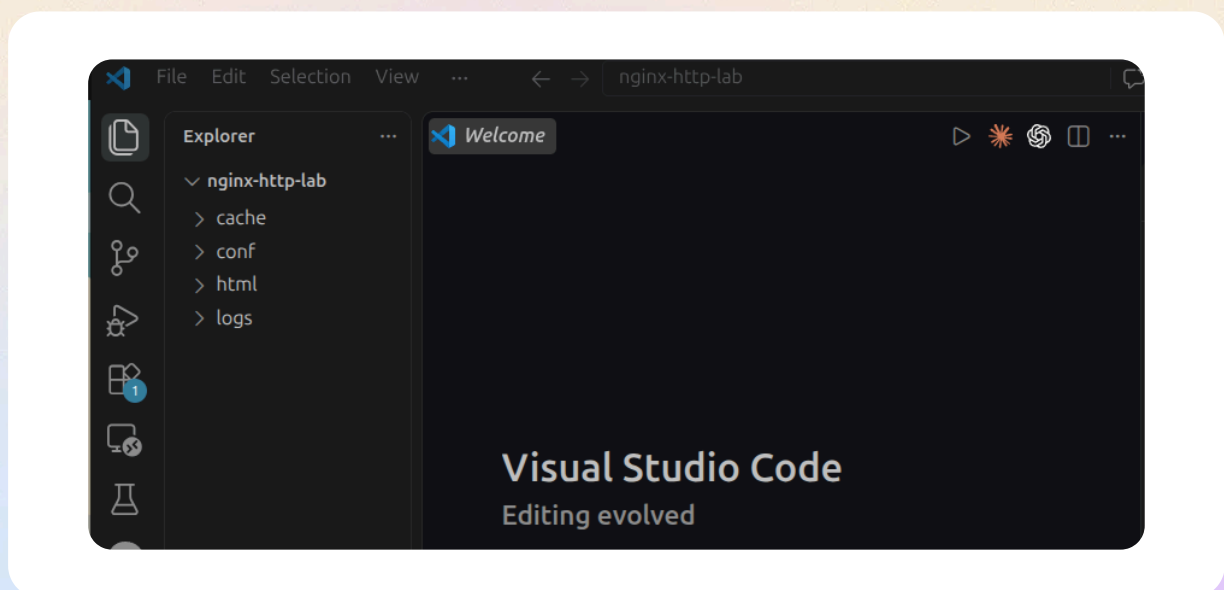
I am building an NGINX gateway that acts as a reverse proxy, and a local gateway for displaying a page on the localhost, and observe a cache hit or hit in the traffic logs.

## Setting up the local NGINX lab environment

In this step, I'm setting up Nginx so that I can create the environment necessary to utilize the tool.

## Confirming the NGINX installation

I verified Nginx was running by checking the nginx version using "nginx -v".



# Extending Load Balancing and Reflecting on the Build

## Measuring weighted load-balancing results

In this project extension, I observed that the two servers shared nginx's default weighting, making requests alternate between them equally.

## Tools and concepts gained

The key tools I used include Nginx, and VS code. Key concepts I learnt include how Nginx can be used as a reverse to balance request among multiple upstream servers.

## Project timeline and challenges

This project took me approximately 50 mins to finish. The most challenging part was setting up the reverse proxy configuration for the backend servers.

## Next steps in my learning journey

I did this project today to learn how to retrieve traffic logs from Nginx and setup Nginx as a reverse proxy.